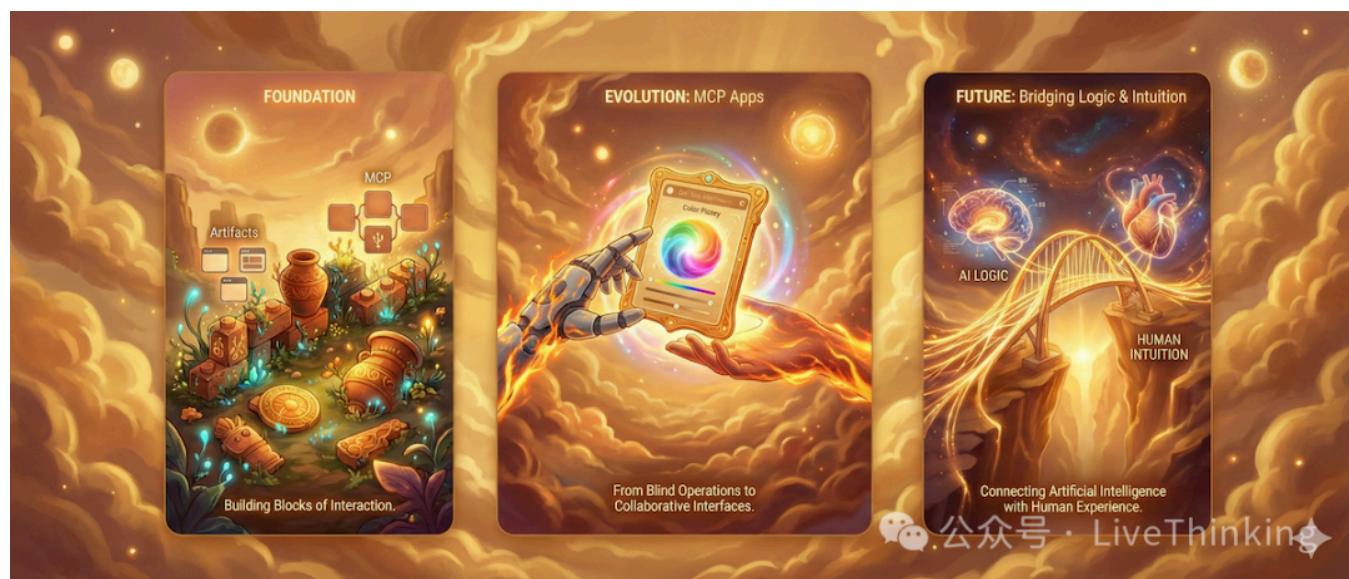


MCP Apps : 当 AI 终于拥有了“界面”

mp.weixin.qq.com/s/x0NEK_3hdKalm5zusODjKg

J杰克



不管外界对 Anthropic 这家公司有着怎样的争议，有一点必须承认，它始终是那个最懂“如何让 AI 与人相处”的专家。

回头这几年，Anthropic 几乎是一步一个脚印地重新定义了我们的交互习惯。

- 先是 Artifacts，第一次把 HTML、SVG 图表从流式对话中“拎”了出来，让信息有了独立的展示窗口；
- 接着是 MCP (Model Context Protocol)，与其说是协议，不如说是 AI 界的“USB 标准”，它终结了 Function Call 的乱战，给大模型装上了通用的手和脚；
- 再到去年年中，Skill 概念横空出世，把单纯的工具调用上升到了逻辑封装的高度，而这个概念在 26年初一下子爆火起来。

每一项技术，都在不知不觉中成为了行业的事实标准。

今天，聊聊 Anthropic 拼图里的最新一块，也是让我非常兴奋的一块 — MCP Apps。它的出现，不仅补全了 AI 应用的一块短板，更让我想起了刚开始做 AI 应用时的往事。

两年前的困局

两年前，ChatGPT 刚刚点燃世界。那时我们做 AI 应用，最头疼的不是模型不够聪明，而是用户不会“说话”。

Prompt Engineering 是个门槛，为了让普通用户能用好 AI，我们当时绞尽脑汁。那时，我们试图开发一种“辅助输入工具”——在聊天框旁边挂一个低代码表单。我们的设想很美好：如在生成周报的场景，就让他填带“本周工作”、“下周计划”两个输入框的表单，而我们再把这些内容拼成高质量的 Prompt 喂给 AI。

但这个项目最终没有做下去。

原因很无奈：缺乏场景感知。那个表单是死的，它是我们预设好的固定逻辑，无法根据对话的上下文灵活变化。而如果要做一个非常灵活的表单，那么就需要有一个通用的表单引擎，而不能是定义好的表单，这个难度就有点大了，而且没有找到能配合的场景。

我们放弃了 GUI（图形界面）的尝试，退回到 System Prompt 的打磨上，试图用更复杂的文字引导来弥补交互的缺陷。

迟来的 Aha Moment

所以，当我第一次看到 Claude 的 MCP Apps 技术文档时，有种被击中的感觉——这不就是我们当年梦寐以求、却做不出来的那个东西吗？！

它完美地解决了那个核心矛盾。MCP Apps 不再是一个挂在旁边的僵硬表单，它被彻底“原子化”了。它平时是隐身的，只有当对话进行到需要它的时候，AI 才会像变魔术一样把它“召唤”出来。

这就好比你现在正在和朋友聊装修。以前，朋友问你“墙面要什么颜色？”，你只能用文字描述“稍微暗一点的红色，带点复古感”，然后两个人像猜谜一样反复确认。现在，朋友直接递给你一本色卡：“你选一个。”

这个“递色卡”的动作，就是 MCP Apps 的精髓。

它意识到，随着模型能力的增强，简单的对话已经不需要辅助了，但在需要复杂参数、结构化配置或者精准确认的时刻，文字依然是苍白的。这时候，直接把 UI 界面请进来，才是最直接、最有效的方法。

不是第四根柱子，而是进化的“手”

从技术视角看，MCP Apps 的设计非常精妙。它没有推翻之前的架构，而是顺势而为。

我们知道 MCP 协议原本有三大支柱：

- • **Resources**: AI 读数据的书
- • **Prompts**: AI 看模版的剧本
- • **Tools**: AI 做事的手

很多人问，MCP App 是不是第四大支柱？其实不是。它是给 **Tools（工具）** 这根柱子装上了一块屏幕。

以前，AI 调用工具是“盲操”。比如要部署一台服务器，AI 必须自己猜测 CPU 核数、内存大小、地区代码，一旦猜错就报错。而有了 MCP Apps，AI 有了更高级的选择：它把决定权交给用户。它弹出一个配置窗口，让用户通过下拉菜单、滑块来精准设置。操作完成后，工具再拿着确定的参数去执行。

这种变化是革命性的。它意味着 AI 交互从“后台批处理”走向了“前台人机协作”。

想象空间瞬间就被打开了。它能做的不只是我们当年想的“填表”：

- 它可以是**结果的可视化**——AI 查完数据，不再丢给你一个 CSV 文件，而是直接弹出一个可交互的图表看板，你能缩放、能排序；
- 它可以是**风险操作的守门员**——当 AI 想要删除文件时，不再只是冷冰冰地问“确认吗？”，而是弹出一个 Diff 对比界面，让你勾选保留哪些内容；
- 它甚至可以是**实时的监控台**——当后台任务在跑的时候，给你展示一个实时跳动的进度条，而不是让你盯着空白屏幕发呆。

极简实战：一个拾色器的诞生

为了做个简单的验证，我马上写了一个最简单的 Demo —— 一个拾色器 (Color Picker)。

这是出于我个人经常遇到的问题。每当我让 AI 帮我写前端代码时，调整颜色总是最痛苦的环节。我总是要切出去找色板，复制十六进制码，再切回来粘贴。

利用 MCP App 的架构，估了一个最简单的 Chat 模拟，将一个色板选择表单做成页面，嵌入到一个 MCP Server 中，然后使用一个更简单的 Chat App 作为 MCP Host，将带界面的 MCP Server 注册成为一个工具。

第一步，在 Chat 输入框输入 `/color` 来发起 MCP 工具的调用，

MCP Web Host

Type `/color` to open the Color Picker App.

Hello! I am a simple Host. Type `"/color"` to test the MCP App integration.

`/color`



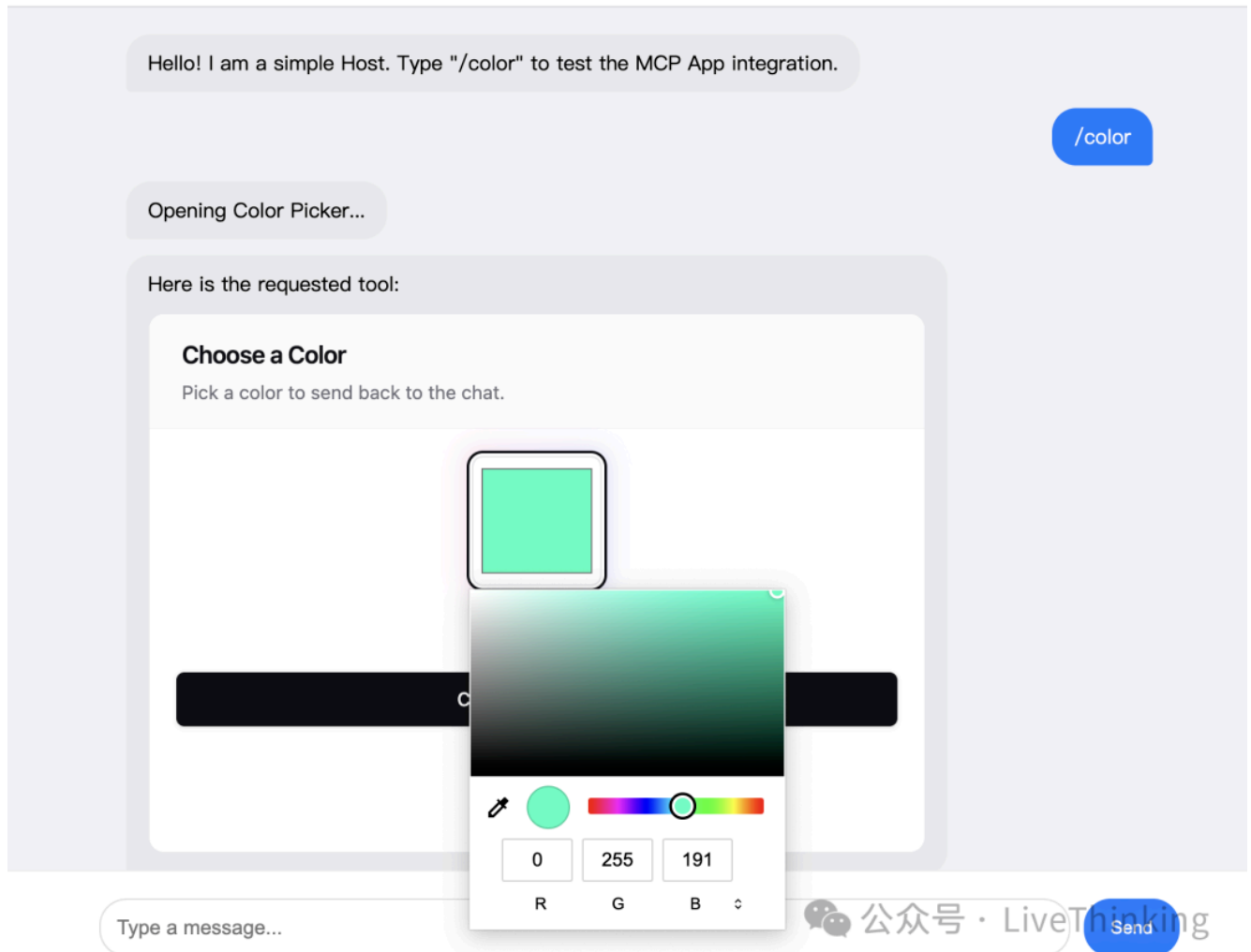
公众号 · LiveThinking

Send

那个熟悉的聊天窗口里，真的直接弹出了我定义的那个精致的小界面，拖动滑块，选中了一个满意的颜色，这时我们面对的不是 AI 返回的大段文字，而是一个操作方便的表单，

MCP Web Host

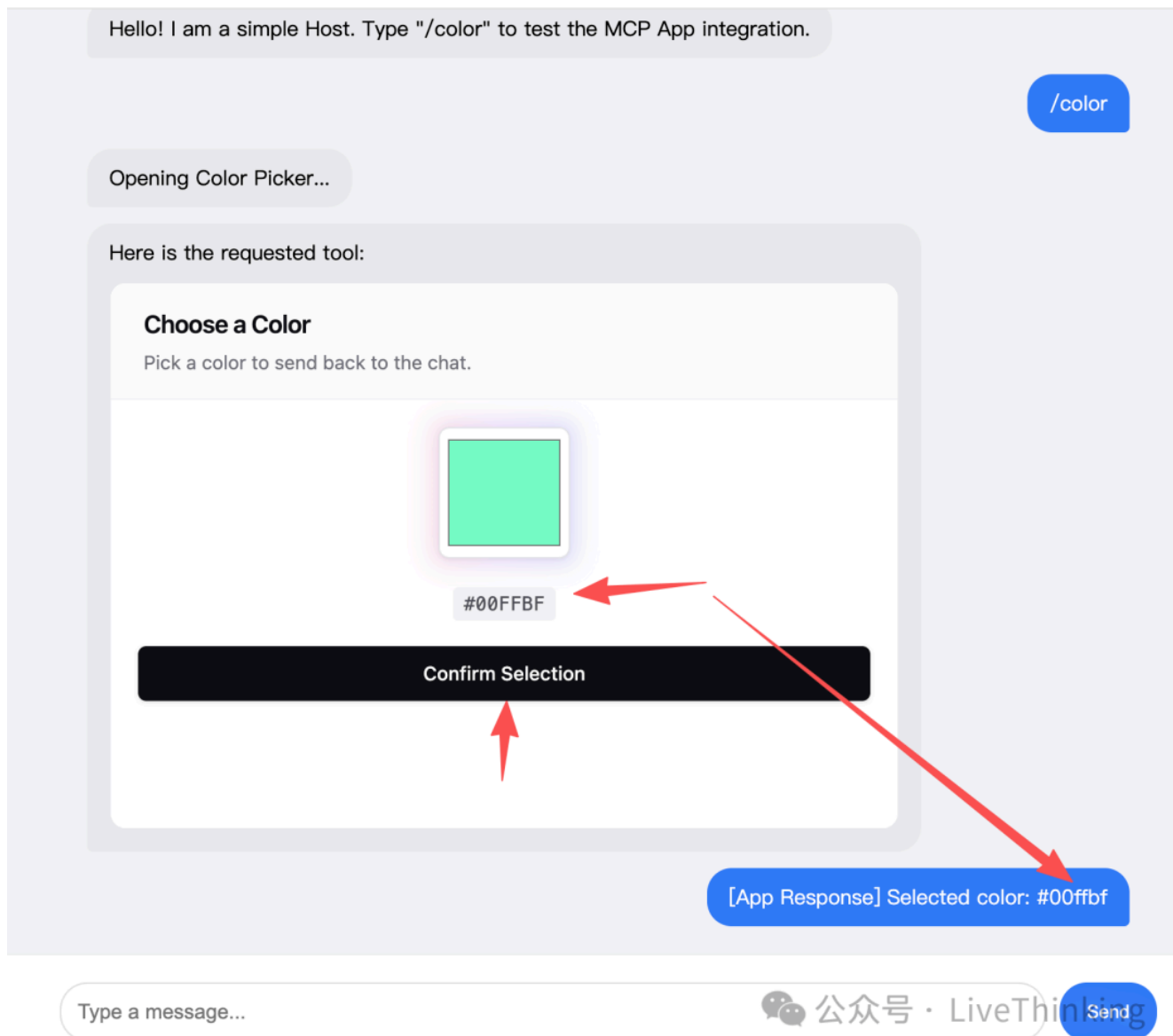
Type `/color` to open the Color Picker App.



在确认选择后，MCP Server 帮我组织一个更加准确的回复，这样让 AI 能更好地理解，

MCP Web Host

Type `/color` to open the Color Picker App.



整个过程顺畅得不可思议。

没有废话，没有歧义，所见即所得。

结语

这就是 MCP Apps 的魅力。

它并没有引入什么惊天动地的新技术，而是用一种极其优雅的方式，把成熟的 Web 生态接驳到了新生的 AI Agent 身上。它让 MCP 不再只是藏在后台的冰冷管道，而是真正走到了前台，成为连接人类直觉与 AI 逻辑的那座桥梁。

这一次，MCP 重回我们的视野，不再仅仅是因为连接数据，更是因为它终于开始在意我们的体验了。

官方：<https://modelcontextprotocol.io/docs/extensions/apps>